Chapter Five: Introduction to Structured Query Language (SQL)

5.1 Chapter objectives

By the end of this chapter, the students should be able to:

- Exhibit understanding what SQL is and can perform
- Demonstrate thorough understanding of data types supported by SQL
- Be able to define and use SQL identifiers
- Use SQL language to create relational database
- Use SQL language to manipulate relational database

5.2 Introduction

SQL stands for Structured Query Language, which is a database for storing, manipulating and retrieving data in Relational DBMS (RDBMS). Some of the popular relational database example are: MySQL, Oracle, MongoDB, SQL Server, DB2, mariaDB, postgreSQL etc.

SQL emerged from the development of the relational model and since then, it has emerged as the standard relational database language. It was the first commercial language introduced for E.F Codd's Relational model of database. Today almost all RDBMS products use SQL as the standard database query language to perform all types of data operations in RDBMS such as

1. CRUD operations(Create, Retrieve, Update & Delete) operations on relational databases.
2. Administrative tasks on database such as database security, backup, user management etc.

5.3 Why SQL can do

i). To create new databases, stored procedures, tables and views

ii). To insert records in a database

iii). To update records in a database

iv). To delete records from a database

v). To retrieve data from a database

vi). Execute queries against a database

vii). Retrieve data from a database

viii). set permissions on tables, procedures, and views

5.4 SQL syntax

SQL follows some unique set of rules and guidelines called syntax. Here, we are providing all the basic SQL syntax.

i). SQL is not case sensitive. Generally, SQL keywords are written in uppercase.

ii). SQL statements are dependent on text lines. We can place a single SQL statement on one or multiple text lines.

iii). You can perform most of the action in a database with SQL statements.

iv). SQL depends on relational algebra and tuple relational calculus.

5.5 Writing SQL commands

SQL statements consist of reversed words and user defined words. Reserved words are those words that have fixed meaning according to SQL. They must be spelt correctly as required and cannot be split across lines. On the other hand, user defined are made up by the user and represents the names of various database objects such as tables, columns, views, indexes etc.

5.6 SQL statement

SQL statements are started with any of the SQL commands/keywords like *SELECT, INSERT, UPDATE, DELETE, ALTER, DROP* etc. and the statement ends with a semicolon *(;)*.

Example of SQL statement:

```
SELECT "column_name" FROM "table_name";
```

Semicolon is used to separate SQL statements. It is a standard way to separate SQL statements in a database system in which more than one SQL statements are used in the same call.

5.7 SQL identifiers

They are used to identify objects in database e.g. table names, views names, and columns.

Rules for defining SQL identities

1. Identifier must start with a letter or _.
2. Identifier cannot contain spaces.
3. SQL identifiers cannot start with a digit
4. SQL keyword cannot be used as identifier name

5.8 SQL data types

The data type of a column defines what value the column can hold: integer, character, money, date and time, binary, and so on. E.g. , in a particular column of a table, if we want to store a string type of data then we will have to declare a string data type of this column.

Data types mainly classified into three categories for every database.

- String Data types
- Numeric Data types
- Date and time Data types

MySQL Data Types (Version 8.0)

5.8.1 String data types

They can contain letters, numbersand special characters

Data type	Description
CHAR(size)	A FIXED length string which can contain letters, numbers, and special characters).The *size* parameter specifies the column length in characters which can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The *size* parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The *size* parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Has Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large OBjects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters

MEDIUMBLOB	For BLOBs (Binary Large OBjects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large OBjects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2,val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

5.8.2 Exact Numeric Data Types

They are used to store numbers without fractions

Data type	Description
BIT(size)	A bit-value type. The number of bits per value is specified in size. The size parameter can hold a value from 1 to 64. The default value for size is 1.
TINYINT(size)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The size parameter specifies the maximum display width (which is 255)
Bool/Boolean	Zero is considered as false, nonzero values are considered as true.

SMALLINT(size)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The size parameter specifies the maximum display width (which is 255)
MEDIUMINT(size)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The size parameter specifies the maximum display width (which is 255)
INT(size)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The size parameter specifies the maximum display width (which is 255)
INTEGER(size)	Equal to INT(size)
BIGINT(size)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The size parameter specifies the maximum display width (which is 255)

5.8.3 Approximate Numeric Data Types

They are used to hold values with decimal points

Data type	Description
FLOAT(size, d)	A floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(p)	A floating point number. MySQL uses the p value to determine whether to use

	FLOAT or DOUBLE for the resulting data type. If p is from 0 to 24, the data type becomes FLOAT(). If p is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(size, d)	A normal-size floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter
DECIMAL(size, d) DEC(size,d)	An exact fixed-point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. The maximum number for size is 65. The maximum number for d is 30. The default value for size is 10. The default value for d is 0.

numeric(p,s)	Fixed precision and scale numbers. Allows numbers from -10^38 +1 to 10^38 –1. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0
smallmoney	Monetary data from -214,748.3648 to 214,748.3647
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807
real	Floating precision number data from -3.40E + 38 to 3.40E + 38

5.8.4 Date time

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(fsp)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(fsp)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(fsp)	A time. Format: hh:mm:ss. The supportec is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit form

value of 0 or 1. The format of declare identifiers of bit type is almost similar to that of character type.

5.8.1 Other data types

sql_variant	It is used for various data types except for text, timestamp, and ntext. It stores up to 8000 bytes of data.
XML	It stores XML formatted data. Maximum 2GB.
cursor	It stores a reference to a cursor used for database operations.
table	It stores result set for later processing.
uniqueidentifier	It stores GUID (Globally unique identifier).

5.9 SQL operators

SQL statements generally contain some reserved words or characters that are used to perform operations such as comparison and arithmetical operations etc. These reserved words or characters are known as operators. We shall look into three types of operators in SQL:

- SQL Arithmetic Operators
- SQL Comparison Operators
- SQL Logical Operators

5.9.1 SQL Arithmetic Operators

Operators	Descriptions
+	It is used to add containing values of both operands
-	It subtracts right hand operand from left hand operand
*	It multiply both operand's values
/	It divides left hand operand by right hand operand
%	It divides left hand operand by right hand operand and returns reminder

5.9.2 Comparison operators

operator	Description
=	Examine both operands value that are equal or not,if yes condition become true.

!=	This is used to check the value of both operands equal or not,if not condition become true.
< >	Examines the operand's value equal or not, if values are not equal condition is true
>	Examine the left operand value is greater than right Operand, if yes condition becomes true
<	Examines the left operand value is less than right Operand, if yes condition becomes true
>=	Examines that the value of left operand is greater than or equal to the value of right operand or not,if yes condition become true
<=	Examines that the value of left operand is less than or equal to the value of right operand or not, if yes condition becomes true
!<	Examines that the left operand value is not less than the right operand value
!>	Examines that the value of left operand is not greater than the value of right operand

5.9.3 SQL Logical Operators

operator	Description
ALL	this is used to compare a value to all values in another value set.
AND	this operator allows the existence of multiple conditions in an SQL statement.
ANY	this operator is used to compare the value in list according to the condition.
BETWEEN	this operator is used to search for values, that are within a set of values
IN	this operator is used to compare a value to that specified list value
NOT	the NOT operator reverse the meaning of any logical operator

OR	this operator is used to combine multiple conditions in SQL statements
EXISTS	the EXISTS operator is used to search for the presence of a row in a specified table
LIKE	this operator is used to compare a value to similar values using wildcard operator

5.10 SQL Built In Functions

SQL provides a number of built in functions and functions that can be used to construct expression as well as provide data type conversion. Built in functions are also used for reformatting data.

Some examples include

i. Curtime() returns the current time in the time zone that is default for the session

ii. Curdate () returns the current date in time zone that is local to the user.

5.11 SQL sub languages

SQL defines following ways to manipulate data stored in an RDBMS.

5.12.1 DDL: Data Definition Language

SQL DDL allows database objects e.g. schemas, domains, tables, views and indexes to be created and destroyed. All DDL commands are auto-committed. That means it saves all the changes permanently in the database.

command	Description
create	to create new table or database
alter	for alteration
truncate	delete data from table
drop	to drop a table
rename	to rename a table

5.12.2 DML: Data Manipulation Language

DML commands are used for manipulating the data stored in the table and not the table itself. DML commands are not auto-committed. It means changes are not permanent to database, they can be rolled back.

command	Description
insert	to insert a new row
update	to update existing row
delete	to delete a row
merge	merging two rows or two tables

5.12.3 Transaction Control Language (TCL)

These commands are to keep a check on other commands and their affect on the database. These commands can annul changes made by other commands by rolling the data back to its original state. It can also make any temporary change permanent.

command	Description
commit	to permanently save
rollback	to undo change
savepoint	to save temporarily

5.12.4 Data Control Language (DCL)

Data control language are the commands to grant and take back authority from any database user.

Command	Description
grant	grant permission of right
revoke	take back permission.

5.12.5 Data Query Language (DQL)

Command	Description
select	retrieve records from one or more table

For this practical, we shall need two tools i.e. software; WAMP server and WAMP MYSQL work bench

The two software can be downloaded from the following locations

https://dev.mysql.com/downloads/file/?id=492434

https://sourceforge.net/projects/wampserver/

- **MySQL Workbench**

  Its a visual database design tool that integrates SQL development, administration, database design, creation and maintenance into a single integrated development environment for the MySQL database system.

MySQL

Int this module, we shall do our lab exercise considering an online store database which has three entities as follows

*Customer(**custID**,name,email,mobileno,city,address,gender,country)*

*Products(**prodID**,name,price,quantity)*

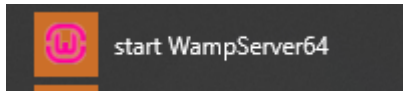
*Orders(**orderID**,prodID,custID,orderdate,quantityordered)*

Data Definition Language (DDL) statements

Object	**SQL Operations**
Schema	▪ create schema ▪ drop schema
Domain	▪ create domain ▪ alter domain ▪ drop domain
Table	▪ create table ▪ alter table ▪ drop table
Views	▪ create view ▪ drop view

These above statements are used to create, change and destroy schema objects.

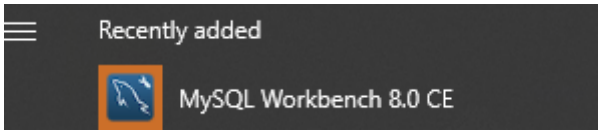
After installing the two programs, click start, choose



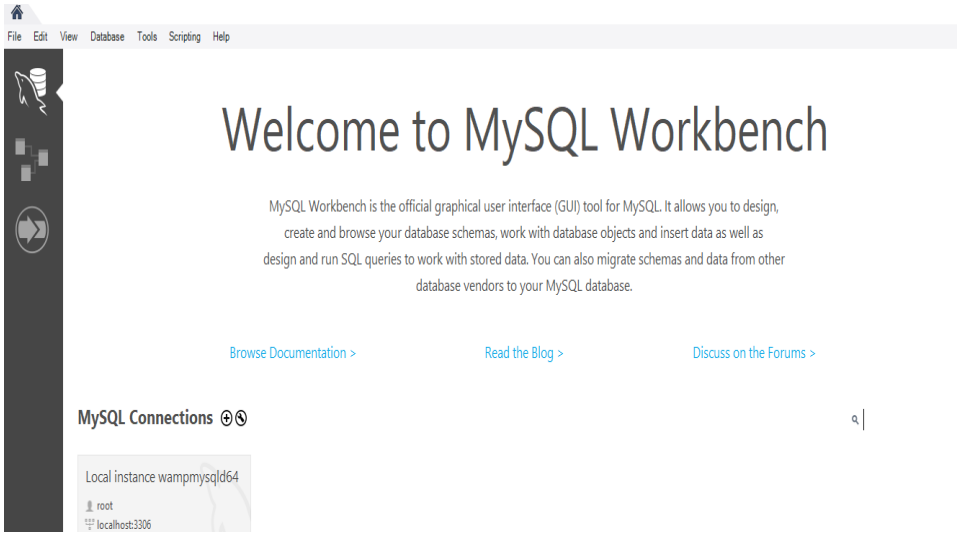
then click “yes” from the dialog that appears

The wamp serve will run in the background and shall appear int the task bar and will appear in color orange before turning into green.

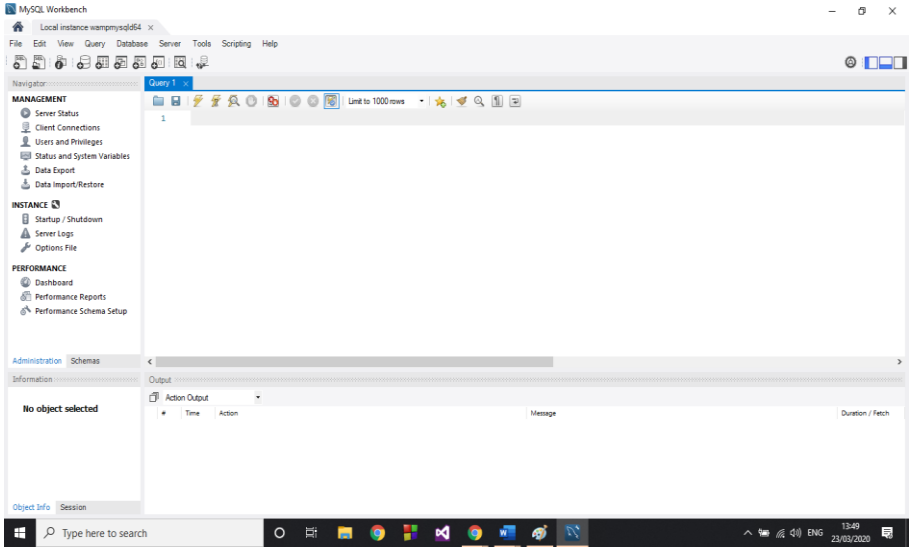
Once the MYSQL icon turns into green, you can click MYSQL workbench from start menu as shown below



After start MySQL workbench, you will get the following page



Click the option "*Local instance Wamp…*" which will open the following window which will be used to create and manipulate your database



We can now start issue SQL statement to manipulate our database.

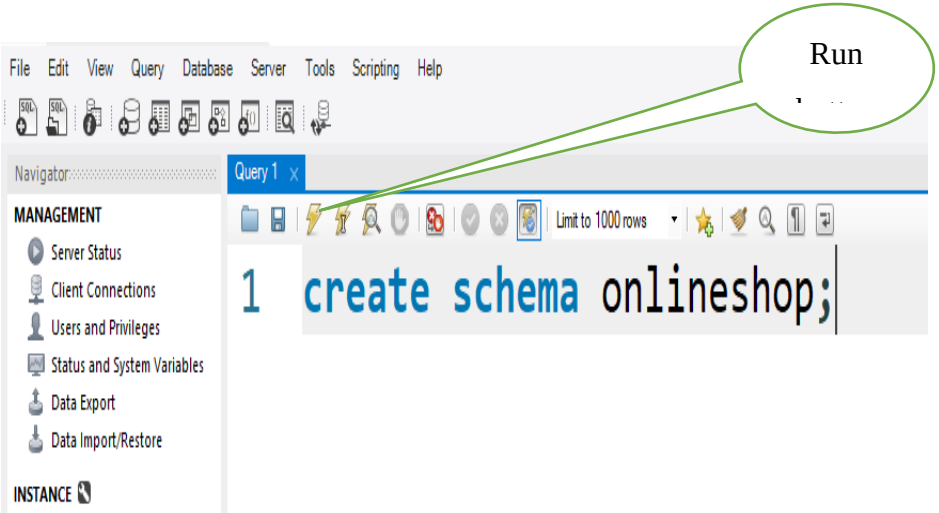
5.12 SQL Database

5.13.1 Creating a database/schema

The first step towards creating a database is to define a schema. To create a schema, we use the following DDL statements.

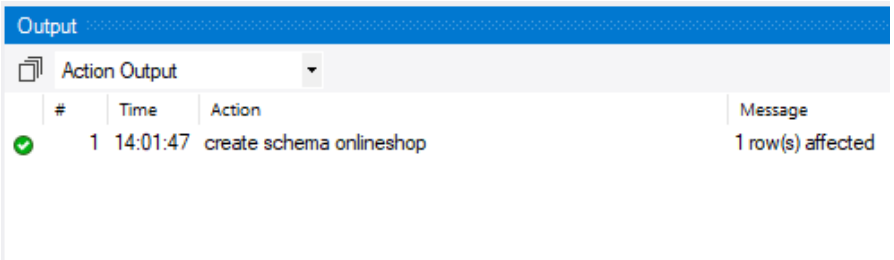
CREATE SCHEMA/DATABASE [name/AUTHORIZATION creator identifier]

In our example, we shall create a database known as online shop

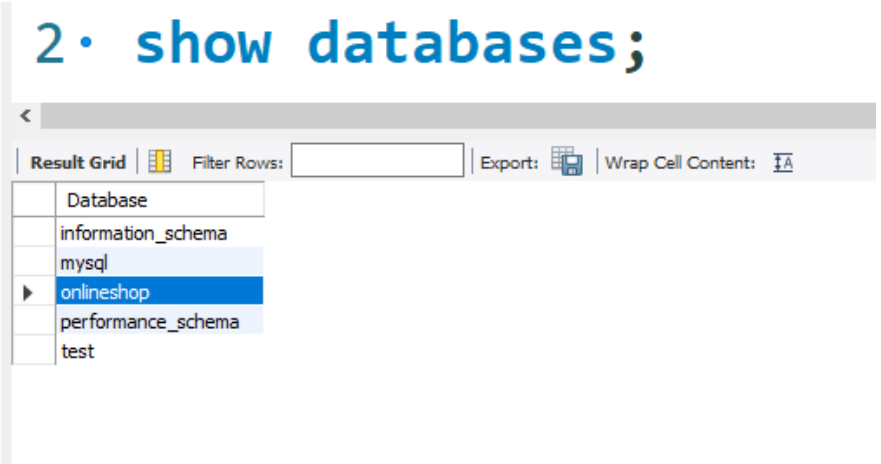


After typing the command, click the run command on the window to execute the statement and create the database.

After running the statement and its successful, you will get a confirmation as shown below. If the feedback has a red color, it means the statement had an error and it wasn't executed. You may have to correct it before executing again.



To confirm that the database was created, you can issue the SQL command show databases which will display all the databases on the current server



You will see that the database has been created and it appears on the list of the databases.

5.13.2 Dropping a schema/database

Schemas can be destroyed using SQL statement ***DROP SCHEMA*** statement which has the following general syntax

DROP SCHEMA/DATABASE name [RESTRICT/CASCADE]

e.g. to drop the above database, we can use the following SQL statement.

```
drop database onlineshop;
```

If we specify restrict, which is the default, the schema must be empty as the operation will fail. If cascade is specified, the operation cascade to drop all objects associated with the schema.

If you delete or drop the database, all the tables and views will also be deleted. So be careful while using this command.

5.13.3 SQL RENAME Database

SQL RENAME DATABASE is used when you need to change the name of your database. Sometimes it is used because you think that the original name is not more relevant to the database or you want to give a temporary name to that database. The rename MySQL database has the follow the following syntax

```
RENAME DATABASE old_db_name TO new_db_name;
```

To rename our database to a new name e.g. *online_db*

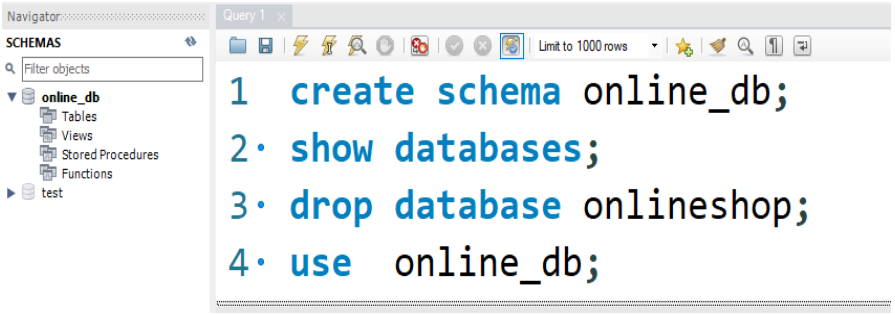
```
RENAME DATABASE onlineshop TO online_db;
```

5.13.4 SQL select database

In MySQL database, you need to select a database first before executing any query on table, view etc. To do so, we use following query:

```
USE database_name;
```

e.g. to select our database online_db, we can use the following SQL code



5.13.5 Integrity enhancement future in SQL

SQL provides facilities for integrity control. Integrity control consists of constraints that we wish to impose indexed to protect the database from becoming inconsistent. These constraints can be defined in the CREATE and ALTER table statements.

5.13.5.1 Required data

Some columns must contain a valid value and hence they cannot be allowed to contain nulls. ISO provides the NOT NULL column specifier in CREATE and ALTER TABLE statement to provide this type of constraint. If we specify a column as NOT NULL, the system rejects attempts to store a null in the column.

To specify o column named student name of student table cannot be null, we define the column as;

```
StudentName van char (40) NOT NULL
```

5.13.5.2 Domain constraints

Every column has a set of allowable values e.g. for gender the domain values can be single string consisting of either 'M' or 'F'. ISO provides two mechanism of specifying domain.

- Use `check clause` statement – which allows a constraint to be defined on a column or the entire table the syntax of check clause is `CHECK (search condition)` *as shown below*

```
gender char NOT NULL CHECK (gender IN ('M', 'F'))
age integer check (age>);
```

- Use `CREATE DOMAIN` statement as shown below

```
CREATE DOMAIN sgender AS CHAR DEFAULT 'M'

CHECK (VALUE IN ('M', 'F'));
```

Which create a domains gender which consists of single character with either value 'M' or 'F'. We can now use *sgender* as data type i.e.

```
Sgender gender NOT NULL
```

5.13.5.3 Entity integrity

The primary must contain a unique not null value for each row. ISO standard supports entity integrity using the *PRIMARY KEY* clause in the CREATE and ALTER TABLE statements as shown below

Regno integer
PRIMARY KEY (regno)

To define a composite key, we specify multiple column names in the *PRIMARY KEY* clause separating each with coma as shown below

PRIMARY KEY (cust_id, product_id);

5.13.5.4 Unique

It is used to ensure uniqueness for any alternate keys in table using. Every column that appears in *UNIQUE* clause must be declared as *NOT NULL*. SQL rejects any INSERT or UPDATE operation that attempts to create a duplicate value within each candidate key (primary key or alternate key)

A good example is presented blow on how to declare unique columns

Regno varchar (9) NOT NULL,
Idno varchar (10) NOT NULL
UNIQUE (Regno, Idno),
Primary key (Regno)

5.13.5.5 Referential integrity

SQL uses the FOREIGN *KEY* clause to indicate a column is a foreign key. A good example is presented below whereby we are making the *sregno* the foreign key and the same is a primary key in *student* table but with a different name, *RegNo*

FOREIGN KEY (Regno) references student (RegNo)

SQL will reject any update or insertion operation that attempts to create a foreign key value in child table without a matching primary key value present in parent/ home table.

5.13.6 SQL BACKUP DATABASE Statement

This SQL statement is used to create a full back up of an existing SQL database. The syntax is as follows:

BACKUP DATABASE databasename TO DISK = 'filepath'

To back up our database to local drive D:, we can use the following SQL statement

```
BACKUP DATABASE online_DB
TO DISK = 'D:\backups\oneline_DB.bak';
```

To back up with differential i.e. only the changes, we use the following SQL statement

```
BACKUP DATABASE online_DB
TO DISK = 'D:\backups\online_DB.bak'
WITH DIFFERENTIAL;
```

The following SQL statement creates a differential back up of the database "online_DB":

5.13 SQL table

Table is a collection of data, organized in terms of rows and columns. A table has a specified number of columns, but can have any number of rows.

Staff ID	Staff Name	SALARY
001	Joe	40,000
002	Peter	90,000
003	Jane	10,00

5.13.1 Creating a table (CREATE TABLE)

Once the database schema has been created, the next step is to create table structures for base relations to be located in the database

SQL CREATE TABLE statement is used to create table in a database. The table should have name the table and define its column and each column's data type. The syntax is as follows

```
create table "tablename"
(
"column1" "data type",
"column2" "data type",
"column3" "data type",
...
"columnN" "data type"
);
```

Let us take an example to create a the customer table with custID as primary key and NOT NULL are the constraint showing that these fields cannot be NULL while creating records in the table.

```
5  create table customer
6  (
7  custid integer primary key,
8  custname text,
9  email varchar(50) unique,
10 mobileno integer    unique,
11 city text,
12 address text,
13 gender char(8),
14 country text
15 );
```

You can verify it, if you have created the table successfully by looking at the message displayed by the SQL Server, else you can use DESC command as follows:

```
1  create schema online_db;
2  desc customer;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
custid	int(11)	NO	PRI	NULL	
custname	text	YES		NULL	
email	varchar(50)	YES	UNI	NULL	
mobileno	int(11)	YES	UNI	NULL	
city	text	YES		NULL	
address	text	YES		NULL	
gender	char(8)	YES		NULL	
country	text	YES		NULL	

5.13.2 SQL drop table

A *SQL DROP TABLE* statement is used to delete a table definition and all data from a table. All data is and table structures forever, so we have to be very careful when using this command. Its syntax is as follows

DROP TABLE "table_name";

DROP TABLE customer;

The above command destroys the entire table structure together with data found in that table.

5.13.3 SQL Rename table

SQL RENAME TABLE syntax is used to change the name of a table. Sometimes, we choose non-meaningful name for the

table. So it is required to be changed. The syntax to rename a table from the database is as follows

ALTER TABLE table_name RENAME TO new_table_name;

ALTER TABLE customer RENAME TO tblCustomers;

The above statements changes the name of the table named customer to tblcustomers

5.13.4 Changing a table definition (ALTER TABLE)

The ISO standard provides an ALTER TABLE statement for changing the structure line of a table once it has been created. The definition of the alter table statement in the ISO standard consists of six options namely;

- Add a new column to a table
- Drop a column from a table
- Set a default for a column
- Add a new table constraints (primary and foreign key)
- Drop a default for a column
- Drop a table constraint (foreign and primary key)
- Rename table

5.13.4.1 Add a new column to a table

The syntax is as follows;

ALTER TABLE TABLENAME Add [column] col-name data type [after col-name][NOT NULL] [UNIQUE];

Example

E.g. ALTER TABLE employee Add column DOB date NOT NULL;

The above SQL statement adds a new column named *DOB* to employee table

5.13.4.2 Drop a column from a table

The drop statement is used to remove/drop a table for a given schema. The syntax is as follows

ALTER TABLE TABLENAME Drop [column] col-name;

Example

ALTER TABLE employee Drop column city;

The above SQL statement removes column named city to employee table

5.13.4.3 Set a default for column

Its used to set a default value for a given column in SQL, we use the following general syntax.

Alter table table_name Alter [column] col_name set default literal;

Example

Alter table employee alter column gender set default 'm';

The above SQL statement adds a default value 'M" for gender. This means if we don't insert a value for the gender column, the default value 'M' shall be added.

5.13.4.4 Change column definition

To change a column name definition i.e. data type and other constraints, we can use the following SQL statement.

```
Alter table table_name Modify [column] col_name column_defination;
```

For example, to change the column email data type from the current data type to varchar(80), we can use the following SQL command

```
Alter table customer Modify email varchar(80) not null;
```

5.13.4.5 Change column new

To change a column name, we can use the following SQL statement.

```
Alter table table_name
Change [column] od_col_name new_col_name column_defination;
```

The SQL statement below changes the customer email column to email address.

```
alter table employee
Change column email emailaddress varchar (80);
```

5.13.4.6 Add a primary key to a table

It's possible to specify a primary key once a table has been created. Maybe the primary key constraint was omitted erroneously when the table was being created.

To do so, we can use the following general syntax

```
Alter table table-name Add primary key (column_name);
```

The following examples adds a primary key to table known as customer

```
Alter table customer add primary key (custid)
```

5.13.4.7 Add a foreign key to a table

Likewise, a foreign key can be specified after the table has been created incase the foreign key constraint was omitted erroneously when the table was being created.

To do so, we can use the following general syntax

```
Alter table table-name Add foreign key (column_name) references parent_table_name (column_name);
```

To make this column the foreign key after the order table has been created, we can use the following statement

Alter table order

Add foreign key (custID) references customer (custID);

5.13.4.8 Remove/drop primary key from a table

It's possible to drop a primary key once a table has been created. To do so, we can use the following general syntax

ALTER TABLE table-name Drop primary key

The following table drops a primary key from the table known as customer

Alter table customer drop primary key

5.13.4.9 Remove/drop foreign key from a table

It's possible to drop a foreign key once a table has been created. To do so, we can use the following general syntax

ALTER TABLE table-name Drop foreign key

The following table drops a foreign key from the table known as customer

Alter table customer drop foreign key

5.13.5 Creating an index (CREATE INDEX)

An index refers to a structure that facilitates accelerated access to the row of a table based on the values of one or more columns. Pressure of an index can significantly improve the performance of a query. Indexes are usually created to satisfy particular search criteria after the table has been in use for some time and has grown in size.

The syntax for creating an index is as follows.

CREATE [UNIQUE] INDEX indexname ON TABLE (col-name)

e.g.

CREATE UNIQUE INDEX regnoindex on student (Regno);

5.13.6 Removing an index (DROP INDEX)

We may decide to destroy/delete/drop an index after creation. We use the DROP INDEX which has the following format.

DROP INDEX indexname;

To remove/ drop the above index, we use can use the following SQL statement

DROP INDEX regnoindex;

5.14 Data Manipulation Language (DML)

SQL has a complete data manipulation language that can be used to modify data residing in a relational database as well as query the database. DML statements include

- INSERT- used to add a new tuple/row of data to a table/relation.
- UPDATE – used to modify existing data in a table/relation.

- DELETE – used to remove rows/tuples from a relation/table.

5.15.1 Adding data to a database (INSERT)

There are three syntaxes of *INSERT* statement in SQL. The first allows a tuple to be inserted into a named relation. Its syntax is as follows;

Insert into table-name [Columnlist] values (datavaluelist)

Where,

- **Table-name** – can be a table name or updatable view
- **Columnlist** – represents lists of attribute(s) names separated by commas. The Columnlist is an optional clause. If it's omitted, SQL assumes a list of all columns in their original CREATE TABLE order. However, if specified, some columns may be omitted from the list.
- **Datavaluelist**-these are the values which shall be stored in the respective columns

The following applies datavaluelist

- The datavaluelist must match the Columnlist.
- There must be a direct correspondence in the position of items in the two lists each that the first item in data value list applies to the first item in Columnlist, the second item in the datavaluelist applies to the second item in the columnlist and so on.
- The data type of each item in datavaluelist must be compatible with the data type of the corresponding column. String data types must be enclosed in single/double quotes

INSERT INTO STUDENT

VALUES (1,"Joe",joe@gmail.com,0719896782,"Nairobi","62-Nrb","Male","Kenya");

Or

INSERT INTO STUDENT(custid,custname,email,mobileno,city,address,gender,country)

VALUES (1,"Joe",joe@gmail.com,0719896782,"Nairobi","62-Nrb","Male","Kenya");

The order of the column names is not vital, but it's advisable to use the order specified during table creation. To insert NULL in an attribute, we use the keyword NULL when specifying the value for that attribute.

```
INSERT INTO STUDENT(custid,custname,email,mobileno,city,address,gender,country)
 VALUES
(1,"Joe",joe@gmail.com,0719896782,"Nairobi",NULL,"Male","Kenya");
```

To insert multiple values in a table, we can use the following

```
INSERT INTO STUDENT(custid,custname,email,mobileno,city,address,gender,country)
 VALUES
(1,"Joe",joe@gmail.com,0719896782,"Nairobi",NULL,"Male","Kenya"),
(2,"phil",phil@gmail.com,0719896783,"Kigalii","678","Male","Kigali");
;
```

5.15.2 Modifying data in the database (updates)

The *UPDATE* statement usually allows modification of existing tuples/ rows. The syntax of the command is as follows;

```
UPDATE TABLENAME
SET col-name 1= datavalue data ,[col-name 2= datavalue 2….],[ Col-name = datavalue n]
WHERE [search condition] ;
```

- Table name is the table/view we are updating
- SET clause specifies the names of column(s) that are being be updated
- *WHERE* clause is optional and if omitted the named attribute is updated for all tuples in the table.

However, if the clause is specified, only matching tuples that satisfy the search condition is updated. For example, the following SQL statement updates all rows of the student fee by increasing their school fee by 20%

```
UPDATE STUDENT _Fee Set fee = fee * 1.02
```

However, if we decide to only increase the fees for first years alone, we may use the WHERE clause as follows;

```
UPDATE student _ fee
SET fee = fee * 1.02
WHERE year _ of _ study = 1;
```

The WHERE clause filter the tuples based on the condition specified i.e. `year_of_study`

To update multiple columns, we specify them in the set statement with each column and its value separated from the next by a comma. String data types are always enclosed in single/double quotes

```
UPDATE STUDENT
SET SNAME = 'Janet', 'DOB= '12/02/2012', 'Paddress='box 62157'  WHERE Regno = 1010102;
```

The above SQL statement modifies the name of student to Janet, date of birth to 12/02/2012 and postal address to 'box 62157 for a student whose registration number is 1010102

5.15.3 Deleting data from the database (DELETE)

The *DELETE* statement allows specific or all tuples to be deleted from a table. The syntax for DELETE command is;

```
DELETE FROM table-name [WHERE search condition];
```

The *table-name* is name of the base relation/update view and the search condition is optional and if omitted, all rows are deleted from the table. Delete command only deletes the contents (data) of a table whenever the search condition is used, only those tuples that meet the search criteria are deleted from the corresponding relation

E.g. delete a tuples of student registration number *09010272*

```
DELETE from student Where Regno=09010272;
```

The where condition finds the rows for registration number 09010272 and deletes the corresponding Tuple(s) from the student relation. To delete all the tuples in a student relation we use the SQL statement

```
DELETE FROM  student;
```

5.15 SQL TRUNCATE TABLE

A truncate SQL statement is used to remove all rows (complete data) from a table. It is similar to the DELETE statement with no WHERE clause. Truncate table is faster and uses lesser resources than DELETE TABLE command.

Its syntax is a follow

```
TRUNCATE TABLE table_name;
```

The following example demonstrates the usage of truncate command

```
Truncate customer;
```

5.16 Chapter summary

```
create schema online_db;
create table customer
(
custid integer primary key,
custname text,
email varchar(50) unique,
```

```
mobileno integerunique,
city text,
address text,
gender char(8),
country text
);
create table product
(
prodID integer primary key,
prodName varchar(100) not null unique,
price double not null,
quantity smallint
);
create table orders
(
prodID integer,
custid integer,
orderid int primary key auto_increment,
orderdate date not null,
quantityordered int not null,
foreign key(prodid) references product(prodid),
foreign key(custid) references customer(custid));
```

5.17 Chapter exercise

5.18 Further reading suggestions